2026-10-25 01:06:04 You

Level 7 — Ansible Galaxy & Collections

69. **Ansible Galaxy**
70. **Installing roles**
71. **Collections**
72. **Installing collections**
73. **Using community modules**
74. **FQCN**
75. **Requirements files**

ChatGPT gpt-6-mini

Ansible Level 7 — Ansible Galaxy & Collections

Goal: Learn how to reuse existing Ansible automation, install community-developed modules, and manage dependencies in real-world projects.

Core idea: Instead of building everything yourself, you can download reusable roles and collections created by the Ansible community or other organizations.

69. Ansible Galaxy

Purpose: Ansible Galaxy is a repository and distribution platform for reusable Ansible content, including roles and collections.

Why use it?

- Reuse automation created by other developers.
- Avoid writing common tasks from scratch.
- Share roles and collections across projects.
- Manage dependencies in a standardized way.

Example use case: Instead of creating your own Nginx role from scratch, you can install a suitable existing role from Galaxy.

Useful commands:

```
</> Bash
```

```
# Search for roles
ansible-galaxy role search nginx

# Install a role
ansible-galaxy role install geerlingguy.nginx

# List installed roles
ansible-galaxy role list

# Search for collections
ansible-galaxy collection search community.docker
```

Remember:

- **Role:** A reusable set of automation tasks and related files.
- **Galaxy:** A platform and command-line tool for discovering and installing reusable Ansible content.

---

70. Installing Roles

Purpose: Download a reusable role and use it in your playbook.

Step 1 — Install a role

Example using the popular Nginx role:

```
</> Bash

ansible-galaxy role install geerlingguy.nginx
```

By default, roles are typically installed in the configured roles path, often `~/.ansible/roles`.

Step 2 — Use the role in a playbook

```
</> YAML

- name: Configure web servers
  hosts: web
  become: true

  roles:
    - role: geerlingguy.nginx
```

Step 3 — Override role variables

```yaml
- name: Configure Nginx
  hosts: web
  become: true

  roles:
    - role: geerlingguy.nginx
      nginx_vhosts:
        - listen: "80"
          server_name: "example.com"
          root: "/var/www/html"
```

The available variables depend on the specific role. Always check its documentation before using it.

Production example: Use a tested community role to install and configure Nginx consistently across multiple servers.

Important: Review a role's source code, maintenance status, supported Ansible versions, and permissions before using it in production.

---

71. Collections

Purpose: Collections package Ansible content into a standardized, versioned namespace.

A collection can include:

- Modules — perform actions.
- Plugins — extend Ansible functionality.
- Roles — reusable automation.
- Playbooks — reusable automation workflows.
- Documentation.

Example collection: `community.docker`

It provides Docker-related modules and plugins.

A collection name usually follows this structure:

```plain text
namespace.collection
```

Example:

```
</> Plain text

community.docker
```

- `community` = namespace.
- `docker` = collection name.

Role vs Collection

Role	Collection
Organizes reusable automation for a particular purpose	Packages multiple kinds of Ansible content
Contains tasks, handlers, templates, files, and variables	Can contain modules, plugins, roles, and more
Example: `geerlingguy.nginx`	Example: `community.docker`

Production example: Install a Docker collection to manage containers and networks, or use a cloud collection to automate cloud resources.

---

72. Installing Collections

Purpose: Download collections so their modules and plugins can be used in playbooks.

Step 1 — Install a collection

```
</> Bash

ansible-galaxy collection install community.docker
```

Step 2 — Verify installed collections

```
</> Bash

ansible-galaxy collection list
```

Step 3 — Use a collection module

Example: Start a Docker container.

```yaml
- name: Manage Docker container
  hosts: docker_hosts
  tasks:
    - name: Start Nginx container
      community.docker.docker_container:
        name: nginx
        image: nginx:stable
        state: started
        ports:
          - "8080:80"
```

Prerequisites: The Docker daemon must be available on the target host, and the required Python Docker SDK must be installed in the Python environment used by the module.

Remember: Installing the collection provides the Ansible module; it does not automatically install or configure Docker itself.

---

73. Using Community Modules

Purpose: Use modules maintained by the Ansible community or other content publishers instead of relying only on built-in modules.

For example, `community.docker` provides modules to manage Docker containers, images, networks, and other Docker resources.

Example — Build a Docker image

```yaml
- name: Build application image
  hosts: docker_hosts
  tasks:
    - name: Build image from Dockerfile
      community.docker.docker_image:
        name: myapp
        tag: v1
        source: build
        build:
          path: /opt/myapp
```

What happens?

1. Ansible connects to the target host.

2. The Docker module uses the Docker API.
3. Docker builds an image from the specified build context.
4. The image is tagged `myapp:v1`.

Built-in vs community modules

Built-in module	Community module
Included with the relevant Ansible installation/package	Usually installed separately through a collection
Example: `ansible.builtin.copy`	Example: `community.docker.docker_container`
Maintained as part of Ansible's built-in content	Maintained by the relevant collection publisher

Production example: Use community modules to automate Docker, Kubernetes, databases, networking, or cloud platforms.

Important: Check the collection's documentation for prerequisites, supported versions, and module-specific behavior.

---

74. FQCN — Fully Qualified Collection Name

Purpose: Unambiguously identify the collection and module or plugin being used.

The standard format is:

```
</> Plain text

namespace.collection.module
```

Examples:

```
</> YAML

# Built-in module
ansible.builtin.copy:
  src: app.conf
  dest: /opt/app.conf
```

```yaml
# Community collection module
community.docker.docker_container:
  name: nginx
  image: nginx:stable
  state: started
```

Breakdown

FQCN	Meaning
`ansible`	Namespace
`builtin`	Collection
`copy`	Module
`community`	Namespace
`docker`	Collection
`docker_container`	Module

Why use FQCN?

- Makes the module's source explicit.
- Avoids ambiguity when different collections provide similarly named modules.
- Improves readability and maintainability.
- Is recommended for production playbooks.

Remember: `copy` is the short module name; `ansible.builtin.copy` is its FQCN.

---

75. Requirements Files

Purpose: Define the roles and collections a project needs so that dependencies can be installed consistently.

There are two commonly used file types:

- `requirements.yml` — can specify roles and collections for `ansible-galaxy`.
- A project may use separate files for roles and collections to make dependencies clearer.

A. Requirements file for roles

requirements.yml

```yaml
roles:
  - name: geerlingguy.nginx
    version: "3.2.0"
```

Install the roles:

```bash
ansible-galaxy role install -r requirements.yml
```

B. Requirements file for collections

requirements.yml

```yaml
collections:
  - name: community.docker
    version: "4.5.0"
```

Install the collections:

```bash
ansible-galaxy collection install -r requirements.yml
```

These are examples of version-pinned dependencies; confirm that the selected versions are available and compatible with your Ansible environment.

C. Combined requirements file

You can also define roles and collections in the same file:

```yaml
---
roles:
  - name: geerlingguy.nginx
    version: "3.2.0"

collections:
```

```
  - name: community.docker
    version: "4.5.0"
```

Install each dependency type using its corresponding command:

```bash
ansible-galaxy role install -r requirements.yml

ansible-galaxy collection install -r requirements.yml
```

Production example: When a new engineer clones the project, they can install its declared dependencies instead of manually finding every role and collection.

Remember: Version pinning improves reproducibility. It does not, by itself, guarantee that dependencies are secure or compatible.

Quick revision — Level 7

#	Concept	Remember this
69	Ansible Galaxy	Discover and distribute reusable Ansible content
70	Installing roles	Download reusable task-based automation
71	Collections	Package modules, plugins, roles, and more
72	Installing collections	Make collection content available to Ansible
73	Community modules	Use modules published outside built-in Ansible content
74	FQCN	Fully identify the source of a module or plugin
75	Requirements files	Declare and install project dependencies

Minimal hands-on practice

- [ ] Install a role using `ansible-galaxy role install`.
- [ ] Install a collection using `ansible-galaxy collection install`.
- [ ] List installed roles and collections.

- [ ] Use a role in a playbook.
- [ ] Use a collection module with its FQCN.
- [ ] Create a `requirements.yml` and install its dependencies.

Interview priority: Understand roles vs collections, Galaxy's purpose, FQCN, community modules, and why production projects pin dependency versions.